Transferência de arquivos via TCP e avaliação de desempenho no emulador CORE

Henrique Campanha Garcia[1], **Loren Pena Rodriguez Lorenzetto**[1]

[1]Universidade Federal de São Paulo (UNIFESP)

henrique.garcia@unifesp.br, rafael.lorenzetto@unifesp.br

Abstract. *This paper presents a C/Linux TCP file-transfer service and its evaluation using the CORE emulator. GET and PUT are tested with four file sizes, two network profiles and up to eight concurrent clients. The experiment completed 528 rounds and 2,288 transfers with matching SHA-256 hashes. Confidence intervals use rounds as the sampling unit. Large files approached the shared link capacity while response times increased with concurrency.*

Resumo. *Este artigo apresenta um serviço de transferência de arquivos em C/Linux e sua avaliação no emulador CORE. GET e PUT foram avaliados com quatro tamanhos de arquivo, dois perfis de rede e até oito clientes simultâneos. Foram concluídas 528 rodadas e 2288 transferências com SHA-256 correto. Os intervalos de confiança usam a rodada como unidade amostral. Arquivos grandes se aproximaram da capacidade do enlace compartilhado, enquanto o tempo de resposta aumentou com a concorrência.*

1. Introdução

TCP oferece um fluxo confiável de bytes, mas não preserva os limites das mensagens da aplicação [Eddy 2022]. Um serviço de transferência precisa, portanto, definir comandos, tamanho do conteúdo e critérios de conclusão. Também precisa tratar operações parciais de socket e impedir que falhas de uma conexão comprometam os demais clientes.

Este trabalho reúne duas etapas: a implementação de um cliente e servidor em C/Linux, com atendimento concorrente via OpenMP, e uma avaliação de GET e PUT no emulador CORE. O objetivo é observar o efeito conjunto do tamanho de arquivo, da concorrência e de dois perfis de rede sobre o tempo de resposta e a vazão útil. A avaliação utiliza processos reais nos nós emulados e verificação de integridade para cada transferência.

2. Implementação do cliente-servidor

A referência funcional foi o par de fontes `tcp_client.c` e `tcp_server.c`. A auditoria identificou um contador de clientes acessado sem sincronização, truncamento do destino antes do fim de PUT, interferência entre gravações do mesmo nome e uma resposta de echo sem a quebra de linha esperada. Os originais foram preservados; a versão revisada mantém os comandos e corrige essas situações.

2.1. Protocolo e tratamento de erros

Os comandos são linhas ASCII encerradas por LF. GET envia o nome remoto; o servidor responde com `OK tamanho` e exatamente a quantidade anunciada de bytes, ou com uma

linha ERR. PUT envia nome e tamanho, aguarda READY, transmite o corpo e aguarda OK. O corpo pode conter qualquer byte, inclusive NUL e LF. O limite é 1 GiB por arquivo; nomes remotos são simples, sem caminhos ou espaços.

As rotinas de comunicação repetem send e recv até completar o tamanho esperado e tratam interrupções EINTR. A leitura do cabeçalho não consome o corpo seguinte. O uso de MSG_NOSIGNAL evita terminar o servidor por SIGPIPE quando um cliente desconecta. Erros que comprometem o enquadramento encerram a conexão; o cliente retorna status não zero quando a operação falha. Há timeout de 15 segundos por espera de socket e limite para estabelecer a conexão.

2.2. Concorrência e sobrescrita

O OpenMP [OpenMP Architecture Review Board 2021] mantém oito trabalhadores em uma região paralela, com ajuste dinâmico desativado. Cada trabalhador aceita e atende uma conexão por vez, sem contador compartilhado. Conexões adicionais podem aguardar no backlog, sujeito aos limites do kernel.

PUT grava em um temporário exclusivo no diretório de armazenamento. Após fechamento bem-sucedido, rename publica o arquivo completo, substituindo o destino existente. Em PUTs concorrentes para o mesmo nome, a última publicação concluída vence. GET mantém o descritor da versão que abriu; o cliente também grava o download em temporário antes de substituir o destino local. Assim, uma transferência interrompida preserva a versão anterior. A publicação é atômica durante a execução normal, sem garantia de durabilidade após queda de energia, pois não se usa fsync em arquivo e diretório.

2.3. Verificação da implementação

A compilação usa GCC, C11, OpenMP e avisos tratados como erros. Passaram 11 testes de integração normais e os mesmos 11 com interposição de sockets, forçando envios de até 4093 bytes, recebimentos de até 257 bytes e EINTR periódico. A suíte cobre arquivos de 0 bytes a 4 MiB, GET/PUT, SHA-256, sobrescrita, cabeçalhos fragmentados, comandos encadeados, desconexões, download truncado, sintaxe inválida, arquivos não regulares e timeout ocioso.

A simultaneidade foi verificada obtendo oito respostas READY antes de enviar qualquer corpo. Também foram testados oito PUTs para o mesmo nome e GET durante um PUT. As verificações da revisão usam novos diretórios de saída, preservando os logs da execução original.

3. Ambiente experimental e topologia CORE

O CORE cria redes virtuais capazes de executar aplicações e protocolos reais sobre recursos do Linux [CORE Project sd]. Foi utilizada a instalação existente do CORE 9.2.0 em um container Docker privilegiado, com Python 3.10. O host era Ubuntu 22.04/WSL2, com kernel 6.18.33.2-microsoft-standard-WSL2. Os nós compartilham CPU, memória, armazenamento e kernel. A identificação da imagem e os detalhes do ambiente foram registrados nas evidências.

A Figura 1 apresenta oito clientes, um servidor, dois roteadores Linux e três switches CORE. O acesso dos clientes usa 10.10.0.0/24; o núcleo, 10.20.0.0/24; e o acesso

do servidor, 10.30.0.0/24. O servidor atende em 10.30.0.10:8080. Os roteadores têm encaminhamento IP e rotas estáticas; clientes e servidor usam seus respectivos gateways de acesso.

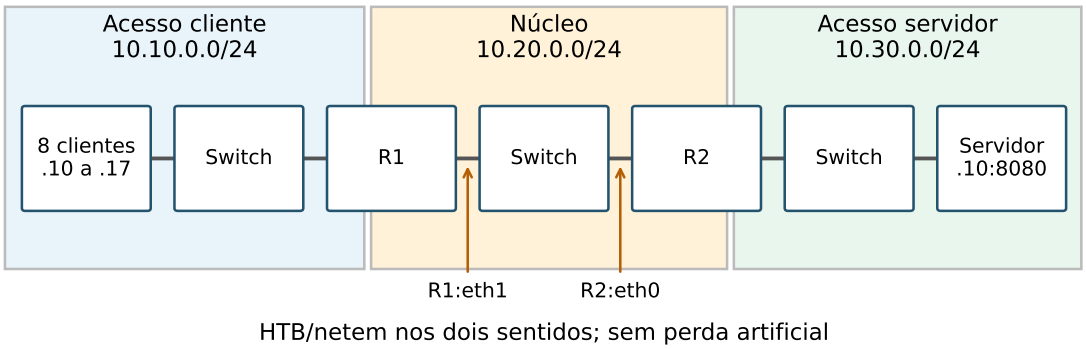


Figura 1. Topologia executada no CORE. O gargalo atua nas saídas R1:eth1 e R2:eth0, cobrindo os dois sentidos de transmissão.

3.1. Perfis e validação da rede

Os acessos Ethernet não recebem limitação artificial adicional. Nas saídas dos roteadores para o núcleo, HTB limita taxa e teto, e netem aplica atraso fixo. O perfil A usa 100 Mbps e 5 ms por direção; B usa 20 Mbps e 30 ms. Os RTT nominais são, respectivamente, 10 e 60 ms. São cenários propostos para o experimento, sem associação universal com uma tecnologia de acesso.

A configuração usa burst de 16 KiB e fila de 10000 pacotes, sem perda artificial. TSO, GSO e GRO foram desativados; o TCP usa CUBIC. Embora não haja descarte artificial, perdas por congestionamento continuam possíveis. Os parâmetros e os contadores de fila foram registrados.

A validação executou ping com dez pacotes, traceroute ICMP e iperf 3.22 nos dois sentidos [ESnet sd]. O iperf usou uma conexão TCP, cinco segundos medidos, um segundo omitido e janela solicitada de 256 KiB. Os resultados recebidos estão na Tabela 1. O traceroute confirmou a passagem por 10.10.0.1, 10.20.0.2 e 10.30.0.10; nenhum pacote do ping foi perdido.

Tabela 1. Validação do caminho cliente–servidor nos dois perfis.

Perfil	RTT (ms)	Ida (Mbps)	Volta (Mbps)	Perdas
A	11,817	95,348	94,995	0/10
B	60,707	19,206	19,085	0/10

4. Metodologia dos experimentos

4.1. Fatores e repetições

Os objetos de 32 e 256 KiB representam tamanhos de miniaturas e imagens comprimidas de uma aplicação Web; 1 e 4 MiB representam anexos ou objetos de armazenamento em

nuvem. São arquivos binários sintéticos determinísticos: o experimento compara custos de comunicação e transmissão, sem reproduzir HTTP, TLS, codecs ou distribuições reais de usuários.

A matriz fatorial cruza dois perfis, GET e PUT separadamente, quatro tamanhos e três concorrências: 1, 4 e 8 clientes. Cada configuração tem 11 repetições, totalizando $2 \times 2 \times 4 \times 3 \times 11 = 528$ rodadas e 2288 transferências individuais. Dentro de cada repetição, as 48 combinações foram embaralhadas com semente 20261105. O arquivo da matriz preserva a ordem.

Um piloto anterior executou GET e PUT em A e B com 256 KiB e quatro clientes: quatro rodadas e 16 transferências corretas. Esse piloto, as validações e os testes de reprodução não entram na amostra principal. Não foram removidos outliers nem descartadas primeiras transferências da campanha. O cache de arquivos não foi limpo; os dados estavam previamente preparados.

4.2. Concorrência, tempo e vazão

Cada cliente é um processo em um nó distinto. O executor fornece uma mesma data futura no relógio monotônico, 150 ms à frente; os processos aguardam essa barreira antes de iniciar a conexão. O cliente C mede início e duração com CLOCK_MONOTONIC. Os instantes são comparáveis porque os nós compartilham o kernel; o relógio de parede apenas identifica a coleta em UTC.

O tempo inclui conexão, comandos, negociação, corpo e conclusão local. PUT termina após receber a confirmação da publicação; GET, após fechar e publicar o destino local. Criação do processo, espera da barreira, geração dos dados e verificação SHA-256 ficam fora da medida. O executor aplica limite externo de 55 segundos ao cliente e watchdog de 60 segundos ao comando do nó.

A vazão útil individual é $G = 8B/(10^6 T)$, em Mbps, com B bytes de corpo e T segundos. A vazão agregada divide a soma dos corpos pelo intervalo entre o primeiro início e a última conclusão da rodada. Ela não é a soma das vazões individuais quando as durações diferem.

4.3. Integridade e retomada

Cada observação registra perfil, operação, concorrência, tamanho, repetição, cliente, tentativa, início, duração, vazão, bytes, status e hashes, além de desvio de início, sobreposição, duração da rodada e vazão agregada. GET compara o destino local à origem; PUT compara o arquivo publicado no servidor à origem do cliente. Na campanha, PUT usa um nome remoto por cliente, evitando que disputas de nomes se tornem um fator adicional.

O CSV é sincronizado em disco antes do marcador de conclusão no journal. A análise só aceita uma tentativa confirmada com todos os clientes da rodada. A retomada foi testada após duas rodadas, sem duplicá-las, e uma execução posterior à conclusão informou que não havia rodadas pendentes. Esse teste pressupõe checkpoint íntegro; não simula queda de energia ou corrupção arbitrária. Falhas com status terminal seriam mantidas como falhas.

4.4. Tratamento estatístico

Clientes da mesma rodada compartilham enlace, filas e servidor. Por isso, a rodada é a unidade de repetição para estimar incerteza. Para tempo e vazão individual, calcula-se a média dos clientes em cada rodada e, então, o IC de 95% entre as 11 médias:

$$\bar{x} \pm t_{0,975;10} \frac{s_{\text{rodadas}}}{\sqrt{11}}. \quad (1)$$

A vazão agregada tem uma observação por rodada. Mediana e desvio padrão entre clientes descrevem a distribuição individual; o CSV também contém o desvio entre médias de rodada. O IC da mediana usa bootstrap percentil com 3000 reamostragens de rodadas inteiras, preservando os clientes agrupados. As taxas usam todas as observações; desempenho considera sucessos. Os IC são marginais, sem correção para comparações múltiplas.

5. Resultados e análise estatística

A campanha principal ocorreu em 5 de novembro de 2026, entre 17:05:21 e 17:22:04 UTC. Foram confirmadas as 528 rodadas e as 2288 transferências, com taxa empírica de sucesso de 100% e de falha de 0%. Todas as verificações SHA-256 corresponderam. As 48 configurações têm 11 repetições e nenhuma observação foi excluída por tentativa incompleta ou substituída.

Todas as rodadas com múltiplos clientes tiveram interseção temporal positiva: a menor foi 33,313 ms, e o maior desvio entre inícios foi 2,038 ms. Esses registros, junto ao teste de oito READY, confirmam que os clientes não foram atendidos sequencialmente.

A Tabela 2 apresenta os tempos para 4 MiB, incluindo média, mediana, DP individual e IC da média. A Figura 2 reúne todos os tamanhos. As estatísticas completas das 48 configurações, inclusive IC das medianas e das vazões, permanecem no CSV resumido.

Tabela 2. Tempo de resposta para 4 MiB, em segundos. O IC da média usa 11 rodadas; mediana e desvio padrão (DP) descrevem as observações individuais.

Perfil	Op.	N	Média	Mediana	DP	IC 95%
A	GET	1	0,396	0,397	0,001	[0,396; 0,397]
A	GET	4	1,301	1,333	0,138	[1,264; 1,339]
A	GET	8	2,602	2,679	0,264	[2,502; 2,702]
A	PUT	1	0,411	0,411	0,003	[0,409; 0,413]
A	PUT	4	1,324	1,382	0,146	[1,273; 1,375]
A	PUT	8	2,543	2,565	0,270	[2,495; 2,591]
B	GET	1	2,024	2,024	0,001	[2,023; 2,024]
B	GET	4	6,626	7,101	0,734	[6,341; 6,912]
B	GET	8	13,030	13,697	1,211	[12,922; 13,138]
B	PUT	1	2,102	2,085	0,055	[2,065; 2,139]
B	PUT	4	6,380	6,495	0,876	[6,159; 6,602]
B	PUT	8	13,422	13,787	0,776	[13,333; 13,510]

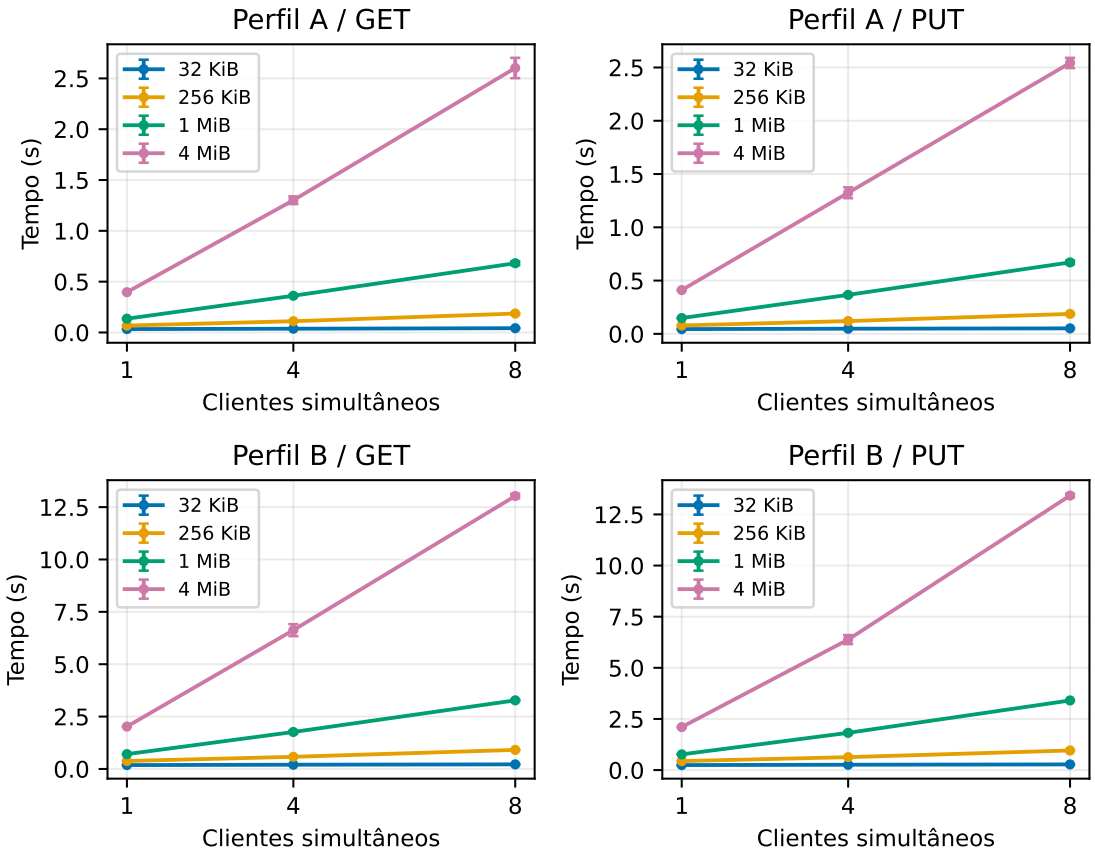


Figura 2. Tempo médio de resposta por operação, perfil e concorrência. As barras representam IC de 95% calculados entre rodadas.

As Figuras 3 e 4 mostram a vazão útil individual e a agregada. Para 4 MiB e oito clientes, a agregada ficou próxima de 94 Mbps em A e 19 Mbps em B. Entre quatro e oito clientes, o ganho agregado variou aproximadamente de 1,4% a 2,1%, conforme a operação e o perfil.

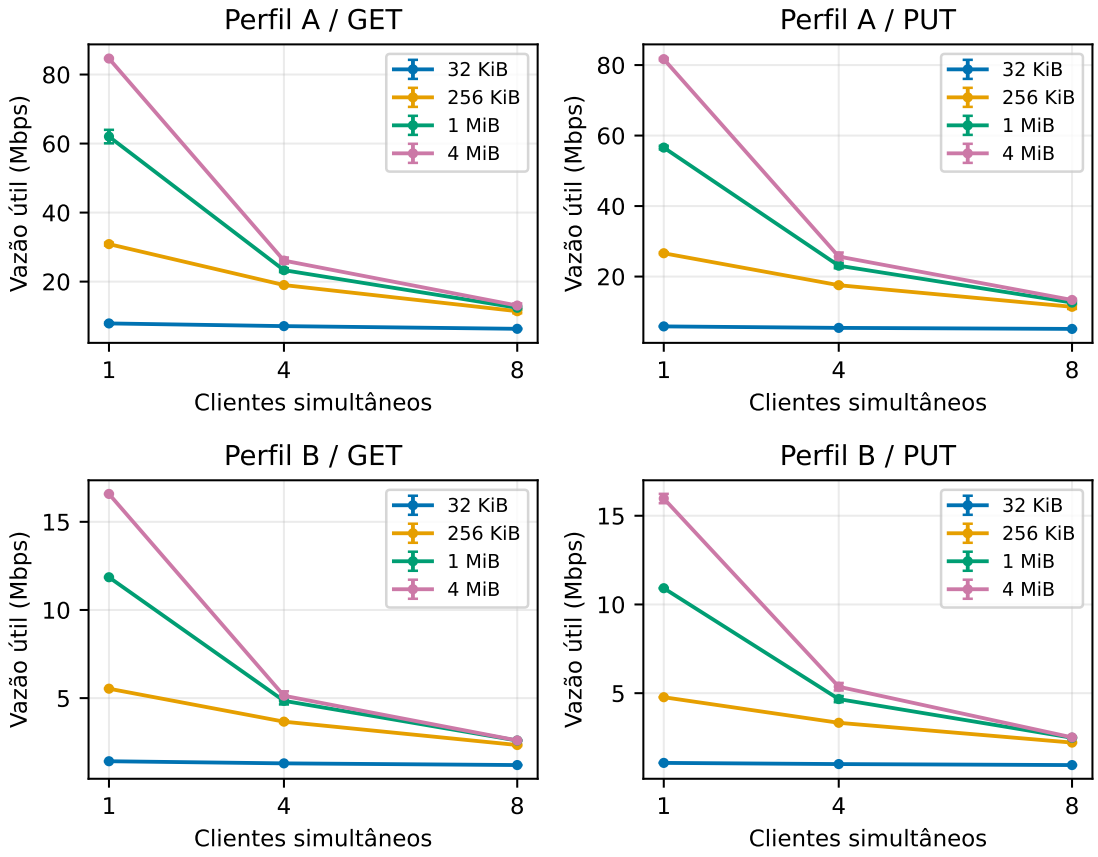


Figura 3. Vazão útil individual. O aumento da concorrência divide a capacidade compartilhada; as barras representam IC de 95% entre rodadas.

Para 32 KiB e um cliente, GET levou cerca de 0,033 s em A e 0,185 s em B; PUT levou 0,045 s e 0,245 s. A Tabela 3 compara razões de médias em tamanhos e concorrências extremos. São comparações descritivas, sem intervalo específico ou teste de hipótese para essas razões.

Tabela 3. Razões entre médias de tempo de resposta. Valores descritivos, sem teste de hipótese ou intervalo específico para as razões.

KiB	N	PUT/GET A	PUT/GET B	B/A GET	B/A PUT
32	1	1,343	1,325	5,543	5,469
32	8	1,222	1,266	5,202	5,387
4096	1	1,036	1,039	5,104	5,117
4096	8	0,977	1,030	5,007	5,279

A Figura 5 apresenta a dispersão individual em 4 MiB, e a Figura 6 mostra a CDF

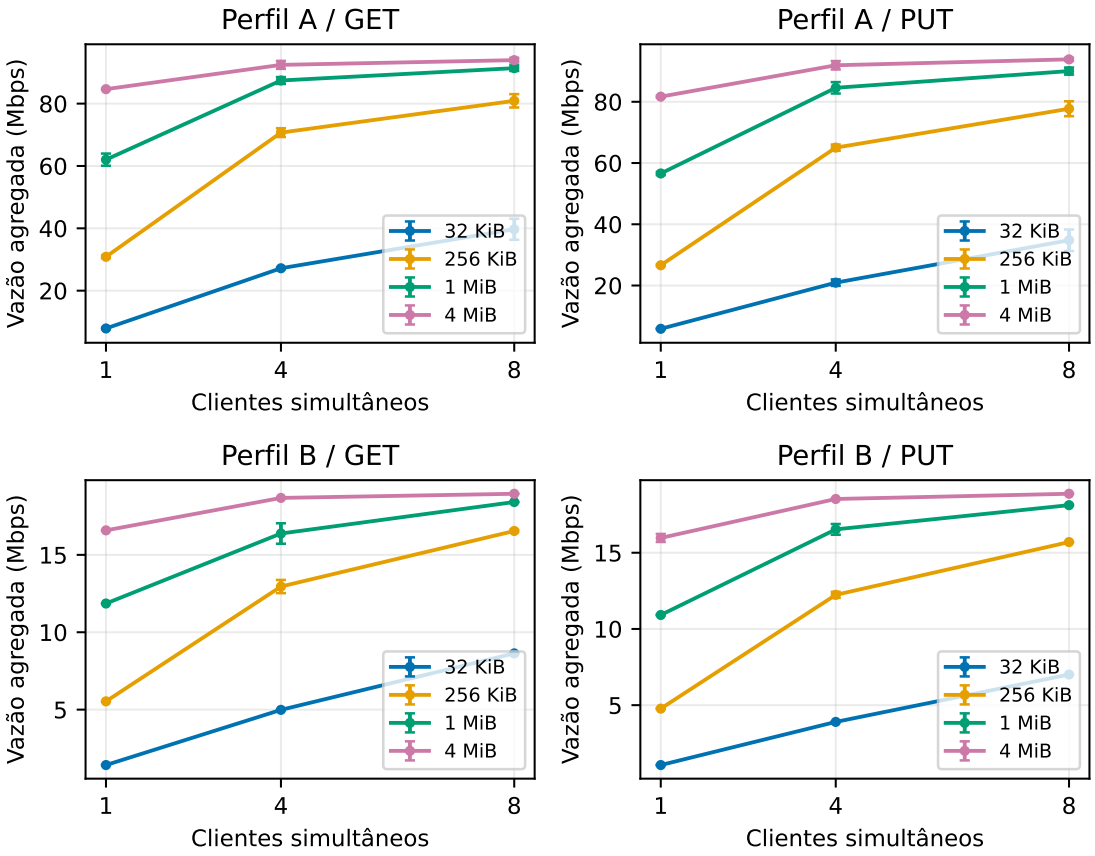


Figura 4. Vazão agregada pelo intervalo completo da rodada, com IC de 95%. A escala de cada perfil acompanha seu limite de capacidade.

empírica para um e oito clientes. Essas visualizações não supõem independência entre clientes da mesma rodada.

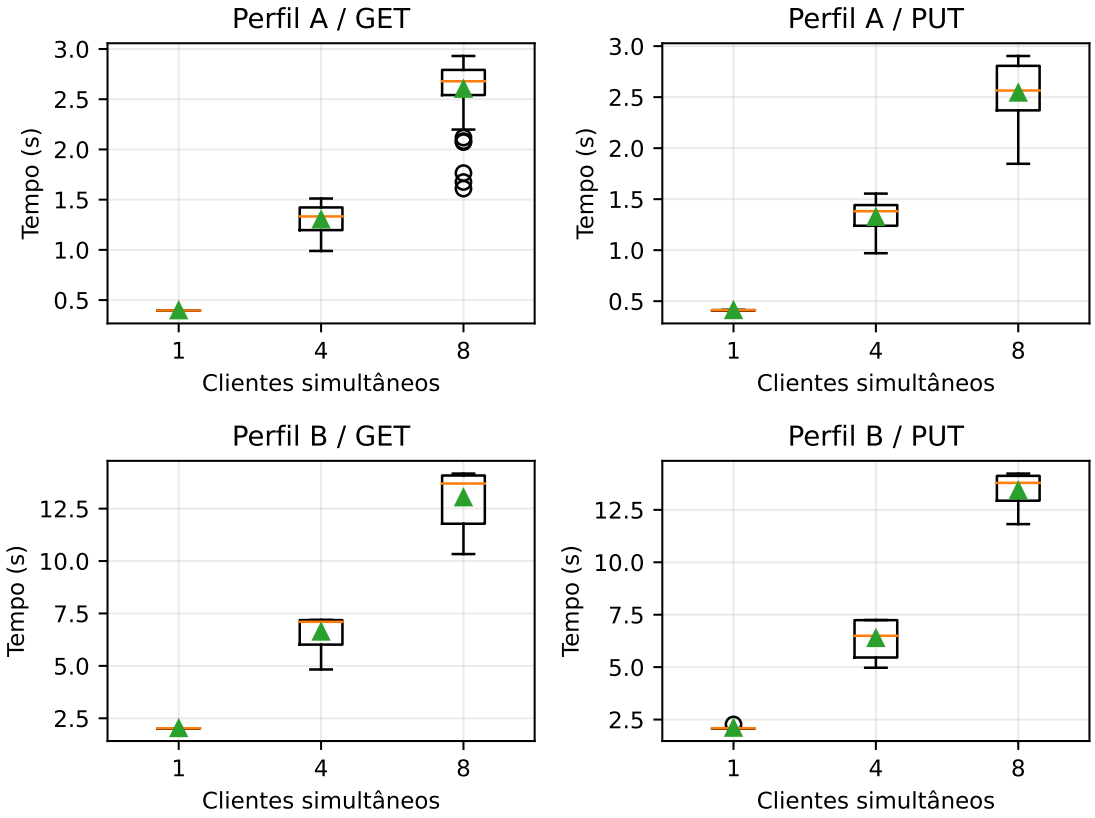


Figura 5. Distribuição do tempo individual para 4 MiB. As caixas e os pontos são descritivos; os marcadores triangulares indicam médias.

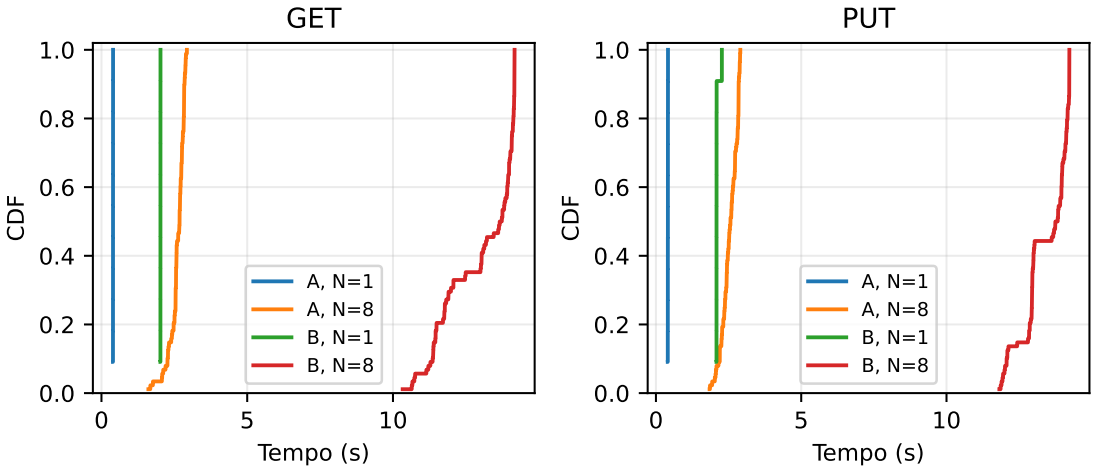


Figura 6. CDF empírica do tempo para 4 MiB. N indica clientes simultâneos; as curvas comparam os perfis A/B e as concorrências extremas.

6. Discussão dos resultados

O pequeno ganho agregado de quatro para oito clientes, acompanhado por aumento do tempo individual, é compatível com saturação do gargalo compartilhado. Em GET de 4 MiB, o tempo passou de aproximadamente 0,396 s para 2,602 s em A e de 2,024 s para 13,030 s em B. Em PUT, passou de 0,411 s para 2,543 s em A e de 2,102 s para 13,422 s em B. Apenas três níveis de concorrência não permitem localizar um ponto exato de saturação.

PUT inclui a espera por READY e pela confirmação final; GET inclui o cabeçalho de resposta e a conclusão da gravação local. Essas diferenças ajudam a interpretar os objetos pequenos, mas não isolam causas. A direção da diferença entre GET e PUT muda em algumas configurações; não se pode concluir que uma operação é sempre mais rápida.

B combina taxa cinco vezes menor e RTT nominal seis vezes maior. Assim, diferenças A/B descrevem o perfil completo, sem separar o efeito de taxa e latência. Objetos grandes sustentam maior uso do gargalo, enquanto os menores são mais sensíveis aos custos fixos das fases de comunicação. Isolar cada fator exigiria outra matriz, variando somente um deles por vez.

6.1. Limitações

Os nós compartilham um host WSL/Docker, sem reserva de CPU ou controle de disco frio. A Ethernet virtual usa atraso fixo, sem jitter ou perda artificial, e a fila de 10000 pacotes não modela um equipamento residencial específico. Os snapshots não incluem acompanhamento contínuo de pacotes, CPU e disco por rodada; portanto, não identificam a causa de cada observação extrema.

Não foram avaliados TLS, autenticação, longa duração ou durabilidade após queda de energia. Onze rodadas oferecem uma estimativa limitada de variabilidade, com possível dependência temporal residual. Zero falhas observadas não significa probabilidade de falha nula fora da amostra.

7. Conclusões

Os testes verificaram GET/PUT binários e vazios, sobrescrita, preservação de versões após falhas e oito atendimentos simultâneos. A campanha no CORE concluiu 528 rodadas e 2288 transferências com integridade confirmada.

Para objetos grandes, aumentar a concorrência de quatro para oito clientes elevou o tempo individual com pouco ganho agregado. A análise usa rodadas para estimar incerteza. Fontes, dados e evidências estão organizados para reprodução, conforme as instruções dos READMEs.

Referências

CORE Project (s.d.). CORE documentation. `https://coreemu.github.io/core/`. Documentação oficial. Acesso em: 5 nov. 2026.

Eddy, W. (2022). Transmission control protocol (TCP). RFC 9293, RFC Editor. Acesso em: 5 nov. 2026.

ESnet (s.d.). Invoking iperf3. `https://software.es.net/iperf/invoking.html`. Documentação oficial. Acesso em: 5 nov. 2026.

OpenMP Architecture Review Board (2021). *OpenMP Application Programming Interface, Version 5.2*. Acesso em: 5 nov. 2026.